

DSA

⇒ Encapsulation :

Bundling data & operations together while controlling access to internal state.

- * getters - sets arguments.
- * setters

⇒ Abstraction :

It is process of eliminating non essential details so that only necessary software features of an entity remains visible to the caller whose caller establishes the clear boundary b/w specification and implementation.

⇒ Interface :-

→ An interface in Java is a pure contract that defines a set of behavior & implementing a class must commit to, interfaces cannot be instantiated directly.

→ A class can implement multiple interfaces by passing a single inheritance restriction of classes.

→ All field declared inside an interface are public, static, void.

→ All methods signature declared are implements public, static.

An abstract class is a class marked with abstract keyword.

- It acts as a partial blueprint intended to be extended by concrete subclasses.
- It has no method body.
- Abstract methods must not contain curly braces.
- Enclosing class constrain
- An abstract method can only exist inside an abstract class or an interface.

Q ⇒ What is instance variable & global variable abstract class v/s interface & local variable

- Mandatory overriding.
- Non-abstract is the first concrete
- The first concrete ^{subclass} extending the abstract parent class must override and provide concrete implementation for all inherited abstract methods.
- Abstract methods cannot be declared as public, private & protected.

Q# Sorter Strategy pattern

Q# Create an abstract class shape that represent geometric shape.

- double area() → abstract method
- double perimeter() → abstract method
- void display() → concrete method

⇒ Inheritance :

- It is a mechanism where in a child class derives state and behaviour from a parent class using the extend keyword.
- It establishes has-a and is-a relationship
 - is-a → derives from parent class
 - has-a → property of parent class.

⇒ Types used in DSA :

- Single → one child extends from one parent.
- multi-level
- hierarchy → class C extends class B and class A.

⇒ Up casting and Down casting :

- this → used for particular class.
- Super → used to access parent class

Student()

{

}

this.name; [this keyword is a
this.age; parameterized constructor]

→ Overloading constructor :

class student string name {

{ student() {

}

student(string name)

this.name;

⇒ Polymorphism :

It came from Greek - many form
It is the capability of a single entity
single entity means method calling
or interface references to exhibit the
different behavior depending on the
context or the underlying

→ Types :-

- static - compile time & overloading
- dynamic - run time & overriding
[binding time], object() → Heap, pointer

⇒ Methods in Static polymorphism :
parameters, types and numbers

→ Compile time has a zero runtime

⇒ Compile time polymorphism :-

Method overloading occurs when a
single class defines multiple methods
sharing the same name but having
different parameter list. Overloading
cannot be achieved solely [normally]
by changing the return type of the
method.

```
public int add (int a, int b)
```

```
{
```

```
    return a+b
```

```
}
```

⇒ Runtime polymorphism :-

Method overriding occurs when a child class provides a specific implementation of a method that already declared in its parent class or interface. When calling an overriding method through a parent reference, Java evaluates the actual ^{object type} ~~time~~ on the heap at run time rather than the reference type.

⇒ Control flow :

Control flow determines the order in which statement execution.

⇒ Sequential execution : It executes line by line to perform addition.

```
int a = 10
```

```
int b = 10
```

```
int sum = a + b
```

```
print(sum)
```

→ Control flow lets the program make decisions and execute different blocks of code based on conditions.

* if statement : It executes the block when the condition is true.

Syntax : if (condition)

{

}

Eg: `int age = 20;`
`{ if (age <= 20)`
`{ System.out.println("Age", age);`
`}`

* if-else statement Provides an alternative path when the condition is false.

Syntax: `if (condition)`
`{`
`Statement`
`}`
`else {`
`Statement`
`}`

Eg: `int marks = 45;`
`if (marks >= 45) {`
`System.out.println("Pass");`
`}`
`else {`
`System.out.println("Fail");`
`}`

* if-else-if ladder :- Checks multiple conditions in sequence only the first two block runs.

Syntax: `if (condition) {`
`}`
`else if (condition) {`
`}`

```
else if (condition) {
```

```
}
```

```
else {
```

```
    Statement
```

```
}
```

Eg:

```
if (int marks = 45;
    if (marks >= 75) {
        System.out.println("Grade A");
    }
    else if (marks >=
        System.out.println("Grade B");
    }
    else {
        System.out.println("Pass");
    }
```

Nested if : An if inside another if

```
if (condition) {
```

```
    if (condition) {
```

```
        Statement;
```

```
    }
```

```
    else if (condition) {
```

```
    }
```

```
    else {
```

```
    }
```

* Switch statement: Instead of writing main if-else we use switch statement.

The switch (expression) is evaluated once

→ The expression is compared to each case if there is a matching the statement is executed.

difference } → The break statement stops the switch after the match.
→ The default statement runs if there is no match statement.

⇒ **Loop** : It executes a block of code based on a condition.

* find odd & even [Even no is 1 to 20]

→ **for loop** : It executes fixed number of iteration.

for (int i = 1 ; i <= 5 ; i++) {
statement.

}

→ **while loop** :

→ **do while**

⇒ **Jump statement** :

→ **break**

→ **continue**

→ **return**

⇒ Methods :

A method is a reusable block of code that encapsulates the single block of responsibility.

^{return type} → method
 void display (int name, int age)
 this / super.

⇒ Recursion : It occurs when method calls itself to solve a problem by working on a smaller instances of that same problem.

→ Java runtime uses a call stack to track method calls every call pushes a new frame [local variables, parameters, return address] onto the stack.

→ When a method returns its frame is popped and control returns to the caller.

```
S  A() {
    B()
    C()
}
```

```
S  B()
    {
```

```
    C()
}
```

```
S  C() {
```

```
    S.out ("Hello")
```

```
}
```

```
main()
```

⇒ **Generics** : It lets classes, interfaces and methods operate on a ^{type} specified by the type parameter specify by the caller giving compile time type safety & eliminating manual casting.

⇒ **Collection framework** :

Set of classes and interfaces that provide ready data structures to store and manipulate group of objects. $\rightarrow \text{list} \langle \text{patient} \rangle = \text{new arraylist} \langle \rangle ();$

→ Collection is an objects that can store multiple element as a single unit.

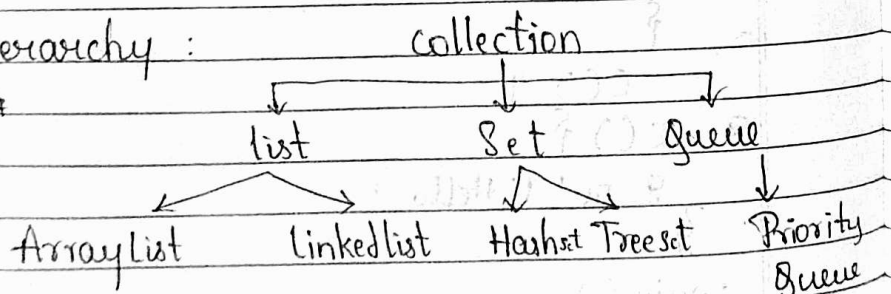
→ Collection provides interfaces defines what ^{data} structure can do.

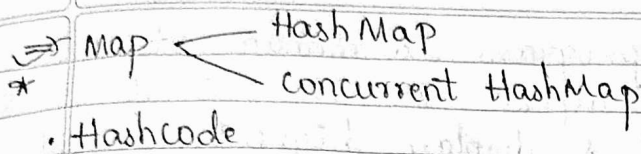
→ Actual classes that implements those interfaces called implementation.

→ Mechanism for traversing collections is called iterators.

* **Difference b/w collection & Collections** → utility class
↳ interface

⇒ **Hierarchy** :





→ Map is a collection framework.

⇒ wrapper class is a class whose object wraps or encapsulates primitive datatypes. [byte, int, char, float]

→ In wrapper class datatypes are written as : <Byte> <Integer> <Character> <Float>

* Primitives-objects ⇒ autoboxing

* objects to primitive ⇒ unboxing

⇒ Lists represents an ordered collection it maintains insertion order and allows duplicates & supports index based access. Elements can be accessed using index.

⇒ ArrayList is one of the most frequently used java collections in DSA.

Syntax : `ArrayList<Integer> classname = new ArrayList<> ( );`

→ It can be backed internally by dynamic array. Unlike, a normal array it can grow dynamically.

Date: _____

16 Write a program to manage student marks arraylist 85 92 78 90 65 88 76 find & display highest, lowest & find average marks, remove all the marks below display after removing. Check whether the mark 90 is still present in the list or not.

⇒ LinkedList :—

It implements three interfaces at once that is list, queue & deque which is why it showed up in some many different roles, internally it is a doubly linked list.

→ Each node stores three things :

- a reference to previous node
- the data
- a reference to next node

→ ArrayList

- Dynamic Array
- Fast
- less overhead
- Insertion $O(1)$

LinkedList

Doubly linked list

Slow

Memory more over

Insertion $O(1)$

⇒ Set Interface :

A set represents a collection of unique elements. It extends the collection interface it is available in the Java.util package. It does not allow duplicate elements

```
Set<Integer> Set = new HashSet<>();
```

→ Methods : `add()`, `contains()`, `remove()`

⇒ HashSet :-

- Use a HashSet when you need fast membership checking and uniqueness.
- It won't order automatically.

⇒ Tree Set :-

Tree set stores unique elements in sorted order.

```
Set <Integer> name = new TreeSet<>();
```

⇒ Queue Interface :-

operations - `offer()` [add]

`peek()` [return first element]

`poll()`

⇒ Map :-

Map stores data as key-value pairs.

Key must be unique and value can be changed.

```
Map <Integer, String> name =  
    new HashMap<>();
```

2. create a simple student result management system where student RollNo is key & student name + marks are value. Perform operations like adding, searching, removing, updating & displaying the result.

⇒ TreeMap :-

Tree Map keeps its ^{keys} ~~tree~~ in sorted order at all the times.

Like, TreeSet Tree map implements a navigable interface with the same family of neighbour finding methods and operating on keys.

→ ceiling key is to find lowest number.

→ floor key is to find highest element.

Eg: map.floorkey(25)

map.ceilingkey(23)

if map.firstkey() → first key

map.headmap(10) → first element

map.tailmap(10) → last element

→ Comparable defines a class single nature ordering implemented inside the class itself.

→ Comparator defines an external swapping ordering useful when you need several different ordering for the same class.

⇒ An Exception :

An exception is an abnormal condition that occurs during the execution of program and disrupts the normal expected flow of instruction.

⇒ Throwable : It is the super class of all objects that can be thrown by java exception. Every error & every exception checked or unchecked ultimately extends Throwable. It provides message methods such as

- getMessage()
- toString()
- getCause() → SQL Exception
- PrintStackTrace()

⇒ Multi-threading : It is the process of executing multiple threads concurrently Thread within the single program

→ Thread is the smallest unit of the execution within the process.

→ There are 3 threads : UI thread, network thread and rendering thread.

⇒ Process versus Thread :

→ A process is a running program with its own independent isolated block of memory that other process cannot directly seen into

Eg : Chrome browser

→ A thread is an execution path within a process.

Date: _____

⇒ Why multithreading:
If everything happens sequentially the application may appear slow. The user is stuck waiting for the download to finish completely before processing even starts and waiting for processing to finish before the UI updates at all even though these are conceptually, fairly independent activities.

→ Creating a thread using extend and implement.

⇒ Thread lifecycle:

run()

demon()

runnable()

⇒ Race condition:

A race condition occurs when multiple threads act on shared mutable data concurrently & the final result depends on the timing or order of their execution.

⇒ Deadlock:

A deadlock occurs when two or more threads wait indefinitely for resources held by each other. If race condition is a case of threads interfering with sharing data. A deadlock is a case of threads interfering with each other.

progress neither thread does anything wrong individually but together they lock the program permanently.

⇒ Synchronization :

It is a mechanism to control the access of multiple threads to share the resources. It is essential in multithreaded environment to prevent thread interference and consistency problems. It ensures that only one thread can access the shared resources at a time.

⇒ Develop traffic signal stimulation using Thread

$$O(m) + O(n) = O(m+n) \text{ [Linear search or two pointers formula]}$$

⇒ Foundation of Complexity :

→ Writing a correct program answers only one program question. A second equally important question is how the efficiently program reaches that answer as the input grows.

→ Complexity analysis is the discipline of answering that 2nd question before you ever run the code.

Eg : Linear Search,
Binary Search

⇒ Two measure the complexity :

- Time complexity
- Space complexity

→ Time complexity is not literally how many seconds does this takes. It describes the growth rate of computational work relative to input size n .

→ Asymptotic analysis :

The time required by an algorithm comes from 3 cases :

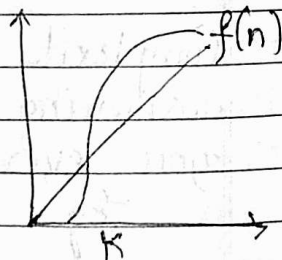
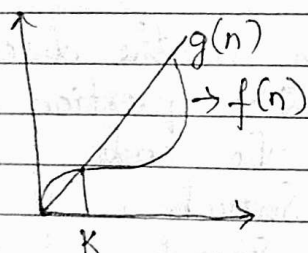
- worst case - when it takes huge time
- best case - when it takes lowest time
- average case - when it takes average time

→ Asymptotic notation :

Based on running time complexity.

- Big-O :- This measures the performance of an algorithm by simply providing order of growth of function.

→ This notation provides upper bound on a fun^c which ensures that fun^c never grows faster than the upper bound.



Big-Oh

- Omega (Ω) notation : It basically describes the best case scenario which is opposite to Big-O notation.
→ It is the formal way to represent lower bound for the algorithm runtime.

- Theta (Θ) notation : It describes average case. It compares both best & worst case.

→ Space complexity : Only fixed number of extra variables are used regardless of n .

1Q6 Given numbers $\text{num} = [2, 7, 11, 15]$ & $\text{target} = 9$. find the indices of two numbers that add up to the target.

2 Write a program to calculate $a^b \bmod m$ using modular exponential. values of a & b maybe very large so the program should avoid directly calculating a^b . o/p : 1024.

⇒

```
for (i = -1; i <= 100; i++) {  
    System.out.println("Hello World!");  
}
```

* Time complexity : $O(n)$

* Space complexity : $O(1)$